Question 1: Signed Binary Arithmetic Using 8-bit 2's Complement

Python Program

```
def binary(n):
    return format(n & 255, '08b')

def subtract(a, b):
    result = a - b
    print("A =", binary(a))
    print("B =", binary(b))
    print("Result =", binary(result))
    print("Decimal Result =", result)

    if result < -128 or result > 127:
        print("Arithmetic Overflow")
    else:
        print("No Overflow")
    print()

print("a) 45 - (+19)")
subtract(45, 19)

print("b) (-35) - (-12)")
subtract(-35, -12)
```

Output

```
a) 45 - (+19)
A = 00101101
B = 00010011
Result = 00011010
Decimal Result = 26
No Overflow

b) (-35) - (-12)
A = 11011101
B = 11110100
Result = 11101001
Decimal Result = -23
No Overflow
```

Manual Calculation

```
a) 45 - (+19)

  00101101   (+45)
- 00010011   (+19)
-----------
  00011010   (+26)

Answer = 26
```

```
b) (-35) - (-12) = (-35) + 12

  11011101   (-35)
+ 00001100   (+12)
-----------
  11101001   (-23)

Answer = -23
```

The answer range for 8-bit signed 2's complement is -128 to +127. Both results are in this range, so there is no overflow.

Question 2: Rotate Operations on 8-bit Binary Numbers

Python Program

```
n = 0b10110110

print("Original:", format(n, '08b'))

print("\nA) Rotate Left Without Carry (ROL)")
x = n
for i in range(2):
    msb = (x >> 7) & 1
    x = ((x << 1) | msb) & 255
    print("Step", i + 1, ":", format(x, '08b'))
print("Final Result:", format(x, '08b'))

print("\nB) Rotate Left With Carry (RCL)")
x = n
carry = 0
print("Initial Carry:", carry)

for i in range(2):
    msb = (x >> 7) & 1
    x = ((x << 1) | carry) & 255
    carry = msb
    print("Step", i + 1, ":", format(x, '08b'),
          "Carry:", carry)

print("Final Result:", format(x, '08b'))
print("Final Carry:", carry)
```

Output

```
Original: 10110110

A) Rotate Left Without Carry (ROL)
Step 1 : 01101101
Step 2 : 11011010
Final Result: 11011010

B) Rotate Left With Carry (RCL)
Initial Carry: 0
Step 1 : 01101100 Carry: 1
Step 2 : 11011001 Carry: 0
Final Result: 11011001
Final Carry: 0
```

Manual Calculation

```
a) ROL (rotate left without carry)

Original: 10110110
Step 1:   10110110 -> 01101101
Step 2:   01101101 -> 11011010

Final Result = 11011010
```

```
b) RCL (rotate left with carry)
```

```
Initial Carry = 0

Step 1: Number = 01101100, Carry = 1
Step 2: Number = 11011001, Carry = 0

Final Result = 11011001
Final Carry = 0
```

Conclusion: Both arithmetic answers are correct and have no overflow. ROL gives 11011010. RCL gives 11011001 with final carry 0.